

# ADR-001 — Canonical REGIME 타입과 Trading registry 매핑

| 항목    | 값                                               |
|-------|-------------------------------------------------|
| 상태    | Accepted                                        |
| 결정일   | 2026-08-20                                      |
| 적용 결정 | D-01, D-02, D-03, D-04, D-09                    |
| 기준 커밋 | 2a70b45adbc443a9c782cf3699e76ec527e2d6be (main) |

## 1. 배경

현재 세 영역은 같은 개념을 서로 다르게 표현한다.

- RegimeSTM은 `TYPE_0~TYPE_4`를 사용한다.
- UI wire 계약은 `type0~type4`를 사용한다.
- TradingSTM은 `LOWER_BB`만 허용한다.

또한 Communication 명세의 `RegimeSTM.run(...)` : `RegimeType`과 인자 없는 `TradingSTM.orderFinished()`는 이미 구현된 두 STM의 안전한 public 계약과 다르다. 이 차이를 코드 통합 전에 해소하지 않으면, 미지원 REGIME이 다른 전략으로 조용히 대체되거나 주문 결과가 잘못된 전이로 전달될 수 있다.

## 2. 결정

### 2.1 Canonical 타입과 wire 값

도메인의 유일한 REGIME 타입은 `TYPE_0~TYPE_4`다. Python 통합 package에서는 `domain/common/enums.py` 한 곳만 이 enum을 소유한다. TypeScript와 transport는 다음의 엄격한 일대일 변환만 사용한다.

| Domain              | Wire               | 표시 의미 |
|---------------------|--------------------|-------|
| <code>TYPE_0</code> | <code>type0</code> | 횡보    |
| <code>TYPE_1</code> | <code>type1</code> | 약상승   |
| <code>TYPE_2</code> | <code>type2</code> | 강상승   |
| <code>TYPE_3</code> | <code>type3</code> | 약하락   |
| <code>TYPE_4</code> | <code>type4</code> | 강하락   |

변환 규칙은 다음과 같다.

- Domain 내부, 저장 schema와 trace에는 대문자 domain 이름을 쓴다.
- HTTP/WS와 UI 계약에서만 소문자 wire 값을 쓴다.
- 대소문자 보정, 숫자 추출, `TYPE_0` 기본값 같은 관대한 변환은 금지한다.
- 표에 없는 입력은 `INVALID_REGIME_TYPE`으로 거부한다.
- `LOWER_BB`는 REGIME 타입이 아니다. `TYPE_0`의 현재 trading transition registry를 식별하는 private registry key로만 유지하고 Phase 1에서 공용 `RegimeType` enum에서 제거한다.

### 2.2 Phase 6 Trading registry coverage 결정

현재 구현과 근거 문서를 기준으로 다음 불변 매핑을 잠근다.

| REGIME | Trading registry                   | 지원 상태       | Registry start Guard      | 상단 BB 정책         | 근거                                                                                                                 |
|--------|------------------------------------|-------------|---------------------------|------------------|--------------------------------------------------------------------------------------------------------------------|
| TYPE_0 | LOWER_BB<br>정확히 109개<br>transition | SUPPORTED   | READY                     | SAFE_TERMINATION | regime_design.md §9의 "기존 30m 횡보 로직",<br>UI_Behavior.md §3의 "Basic Iterative 횡보장 조건", 현재 유일한 30m 구현인 하단 BB registry |
| TYPE_1 | 없음                                 | UNSUPPORTED | UNSUPPORTED_TRADING_LOGIC | 없음               | 약상승 30m Event-Action Table/registry 없음                                                                             |
| TYPE_2 | 없음                                 | UNSUPPORTED | UNSUPPORTED_TRADING_LOGIC | 없음               | 상위 gate 설명만 있고 강상승 30m Event-Action Table/registry 없음                                                              |
| TYPE_3 | 없음                                 | UNSUPPORTED | UNSUPPORTED_TRADING_LOGIC | 없음               | 약하락 30m Event-Action Table/registry 없음                                                                             |
| TYPE_4 | 없음                                 | UNSUPPORTED | UNSUPPORTED_TRADING_LOGIC | 없음               | 하위 gate 설명만 있고 강하락 30m Event-Action Table/registry 없음                                                              |

TYPE\_0 -> LOWER\_BB는 기존 문서의 이름이 완전히 일치해서 발견된 매핑이 아니라, 위 세 근거를 함께 적용해 Phase 0에서确定的 architecture decision이다. 향후 근거가 달라지면 이 ADR을 대체하고 Communication 명세와 Event-Action Table을 먼저 변경한다.

Phase 6에서 lower-BB registry의 G-07은 미구현 상단 전략으로 인계하지 않고 UpperBandPolicy.SAFE\_TERMINATION을 적용하도록 확정했다. realtime\_price >= upper\_band인 경우 다음 세 branch는 배타적이며 pending 주문을 가장 먼저 처리한다.

1. pending\_order\_id != None이면 STOPPING으로 전이하고 reason UPPER\_BAND\_SAFE\_TERMINATION으로 취소를 요청한 뒤, 같은 주문 ID를 stop\_after\_reconciliation = True로 조정한다. 이 branch에서는 ForceSellAll을 즉시 요청하지 않는다.
2. pending 주문이 없고 authoritative context.position.quantity > 0이면 STOPPING으로 전이하고 전략 평가를 취소한 뒤 ForceSellAll을 요청한다. 후속 완료·실패는 기존 G-06F/G-06R로 처리한다.
3. 둘 다 없으면 LOGIC\_TERMINATED로 즉시 전이하고 lower event와 Case Context를 정리한 뒤 trading\_phase = TERMINATED, session 평가 취소, runtime 종료를 순서대로 요청한다.

이 계약은 상단 BB 매매 전략을 추가한 것이 아니라 기존 STOP 처리를 재사용해 하단 BB session을 안전하게 종료하는 정책이다. 따라서 TYPE\_0의 registry coverage와 start Guard는 READY이며, 조용한 no-op이나 인계 fallback은 없다.

미지원 타입도 추천·표시·선택할 수는 있다. 다만 UI는 지원 상태를 함께 표시하고, TradingController.fetchSelectedTradingLogic(...)은 TYPE\_1~TYPE\_4에 인스턴스를 만들지 않고 UNSUPPORTED\_TRADING\_LOGIC으로 거부한다. 어떤 경우에도 TYPE\_0이나 LOWER\_BB로 fallback하지 않는다. 새 REGIME을 지원하려면 해당 30m Event-Action Table, 상태도, registry ID 목록과 경계 테스트가 먼저 있어야 한다.

Registry의 READY는 선택한 전략 구성이 완전하다는 domain 신호이지 즉시 production 주문을 허용한다는 의미가 아니다. Phase 7의 session 시작 orchestration은 fake mode에서만 command\_enabled = true로 연결한다. disabled, testnet, live는 Phase 7에서 fail closed이며 실제 Gateway 주문은 Phase 8 이후의 실행 mode gate를 별도로 통과해야 한다.

Phase 7 application 경계에서 사용자 선택은 RegimeController.set\_regime\_type(regime\_type, \*, command\_id, expected\_version)으로만 commit하고 typed TradingLogicSelectionResult를 반환한다. start/stop도 같은 command ID

와 expected Context version 계약을 사용해 typed `TradingSessionResult`를 반환한다. 이 metadata는 HTTP transport가 생성·검증해 application Operation에 전달하며 추천 REGIME 갱신 경로에서는 사용하지 않는다.

## 2.3 RegimeSTM public 계약

RegimeSTM의 canonical operation은 다음 하나다.

```
handle(event : RegimeEvent, context : RegimeEvaluationContext?) : RegimeSTMResult
```

- 시작 event와 4H 마감 event는 첫 microstep에서 `StartRegimeEvaluation`을 반환한다.
- RegimeController는 같은 `evaluation_id`의 Context를 검증한 뒤 `EVALUATION_READY`를 다음 microstep으로 전달한다.
- 두 번째 결과의 `ApplyRecommendedRegime`을 수행한 뒤에만 `recommendedRegime`을 갱신한다.
- `recommendRegime(...)` : `RegimeType`은 Controller façade다. STM의 `run(...)`을 호출하는 별도 판정 경로가 아니며 반드시 위 두 microstep을 사용한다.
- 추천값은 사용자 `selectedRegime`을 바꾸지 않는다.

## 2.4 TradingSTM 주문 결과 계약

TradingSTM의 canonical decision operation은 다음과 같다.

```
handle(event : TradingEvent, context : TradingContextView) : TradingSTMResult
```

Communication 호환 operation은 다음과 같이 구체화한다.

```
orderFinished(
  event : TradingEvent,
  context : TradingContextView
) : TradingSTMResult
```

`orderFinished`는 `handle`의 검증 adapter일 뿐 별도의 상태 추론 경로가 아니다. 다음의 구체적인 정규화 event만 허용한다.

- `CASE_B_POSITION_OPENED`, `CASE_B_BUY_FAILED`
- `CASE_B_SELL_FILLED`, `CASE_B_SELL_FAILED`
- `CASE_C_POSITION_OPENED`, `CASE_C_BUY_FAILED`
- `CASE_C_SELL_FILLED`, `CASE_C_SELL_FAILED`
- `FORCE_SELL_FINISHED`, `FORCE_SELL_FAILED`

Controller가 실제 fill을 Position에 반영하고 거래 이력 저장을 완료한 뒤에만 성공 event를 전달한다. 인자 없는 호출이나 pending Context로 성공·실패를 추론하는 구현은 금지한다.

## 2.5 실행 중 REGIME 변경

첫 release에서는 active TradingSTM hot-swap을 금지한다.

- `tradingPhase`가 `IDLE` 또는 `TERMINATED`일 때만 선택 REGIME을 바꿀 수 있다.
- `ENTRY_ORDER_PENDING`, `EXIT_ORDER_PENDING`, `STOPPING`, `RECONCILIATION_REQUIRED`를 포함해 세션이 active이면 `TRADING_ACTIVE`로 거부한다.
- 거부 시 기존 `selectedRegime`, STM 인스턴스와 Context는 바뀌지 않는다.
- 사용자는 정상 stop 완료 뒤 REGIME을 선택하고 새 session을 시작해야 한다.
- UI 문서의 실행 중 즉시 적용 표현은 이 결정으로 대체되며, UI 연결 Phase에서 동일 오류와 안내를 반영한다.

## 3. 대안과 기각 이유

- 다섯 REGIME을 모두 **LOWER\_BB**로 연결: 서로 다른 30m 매매 규칙을 추측하므로 기각한다.
- **LOWER\_BB**를 여섯 번째 REGIME으로 유지: 4H 분류와 trading registry 식별자를 혼합하므로 기각한다.
- 실행 중 STM 교체: pending 주문과 Position owner의 의미가 바뀔 수 있어 기각한다.
- 인자 없는 **orderFinished()**: 결과 종류와 대상 전략을 안전하게 식별할 수 없어 기각한다.

## 4. 구현 및 검증 의무

- ☒ 다섯 domain/wire 값의 일대일 표가 확정되었다.
- ☒ **TYPE\_0**은 **SUPPORTED/LOWER\_BB/READY/SAFE\_TERMINATION**, **TYPE\_1~TYPE\_4**는 **UNSUPPORTED**/registry 없음/**UNSUPPORTED\_TRADING\_LOGIC**으로 고정되었다.
- ☒ lower-BB registry의 소속과 근거가 기록되었다.
- ☒ 두 STM의 canonical signature가 실제 구현과 일치한다.
- ☒ active session의 REGIME 변경 거부가 확정되었다.
- ☒ Phase 1에서 중복 Python **RegimeType**을 **domain/common/enums.py** 한 enum으로 통합했다.
- ☒ Phase 6에서 **TYPE\_0** 상단 BB gap을 안전 종료 정책으로 닫고 mapping/fallback gate를 코드-테스트로 고정했다.
- ☒ UI snapshot과 선택 화면에 다섯 REGIME 지원 상태를 반영하고 미지원 start를 차단했다.
- ☒ Phase 7에서 versioned selection/start/stop application Operation, session 시작 orchestration과 active 변경의 **TRADING\_ACTIVE** 오류를 완성했다.

## 5. Baseline 증거

2026-08-20 20:39 KST, 기준 커밋 **2a70b45**에서 실행했다.

| 영역           | 명령                                                             | 결과                 |
|--------------|----------------------------------------------------------------|--------------------|
| RegimeSTM    | <b>PYTHONPATH=src python3 -m unittest discover -s tests -v</b> | 31/31 통과           |
| TradingSTM   | <b>PYTHONPATH=src python3 -m unittest discover -s tests -v</b> | 24/24 통과           |
| UI           | <b>./node_modules/.bin/vitest run --reporter=dot</b>           | 26 files, 88/88 통과 |
| UI typecheck | <b>./node_modules/.bin/tsc -b --pretty false</b>               | 통과                 |
| UI build     | <b>./node_modules/.bin/vite build</b>                          | 274 modules, 성공    |

검증 과정에서 production source 동작은 변경하지 않았다.

### Phase 7 완료 증거

2026-08-21 KST, 시작 커밋 **3e799e126bbb87a88b1e3a522c8f1a7015e5a8e0**에서 mutable **TradingContext** 초기화 뒤 session 전용 **TradingSTM.run()**이 정확히 한 번 호출되는 start 흐름을 구현했다. active 상태에서 REGIME 변경은 **TRADING\_ACTIVE**로 거부되고 선택값, STM 인스턴스와 Context가 모두 유지된다. selection, start와 stop은 **command\_id.expected\_version**을 검증하고 typed 결과를 반환한다. 정상 종료 뒤 같은 REGIME도 새 selection command 전에는 restart할 수 없다. 통합 backend **unittest** 325개와 UI Vitest 155개가 통과했으며, 실제 Binance 주문은 호출하지 않았다.